0287. Find the Duplicate Number

m | Category: Linked List | LeetCode: https://leetcode.com/problems/find-the-du

Problem

- Nums has n+1 integers in [1, n]; exactly one value repeats (possibly many times). Find it without modifying nums and with O(1) extra space.

Intuition

- Treat i → nums[i] as a "next pointer". Index 0 is never a target, so it is the list head.
- Two indices point to the duplicate value, so the functional graph has a cycle whose entrance is the duplicate.
- Floyd's algorithm finds the cycle entrance (Linked List Cycle II).

Brute Force

- HashSet (O(n) space) or sort (modifies input).

Optimal Approach

- Slow = nums[0], fast = nums[nums[0]]; advance until they meet.
- Reset slow = 0; move both one step until equal. That value is the duplicate.

Worked Example

```
nums = [1,3,4,2,2].

path from 0: 0 -> 1 -> 3 -> 2 -> 4 -> 2 -> 4 ...   (cycle 2,4)
phase 1: slow/fast meet inside the cycle (at 4)
phase 2: slow from 0: 1,3,2 ; fast from 4: 2,4,2 -> both at 2
answer 2
```

Java Solution

```java
public class FindTheDuplicateNumber {
    public int findDuplicate(int[] nums) {
        int slow = 0, fast = 0;
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);
        slow = 0;
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Why phase 2 works: distance head→entrance equals distance meeting point→entrance (mod cycle length).
- Binary search on value counts (O(n log n)) is an acceptable alternative.